

Parcial de Python — v4

Condicionales · for/while · def & lambda · Ciencia de Datos (Nivel avanzado)

⌚ 100 min
📄 20 preguntas
🎯 10 puntos

Instrucciones generales

- ▶ Individual, en computador
- ▶ Solo Python estándar
- ▶ Guarde cada respuesta en un .py
- ▶ Sin conexión a internet
- ▶ Cada pregunta vale 0.5 pts
- ▶ Se evalúa lógica, estilo y casos borde

λ = uso obligatorio de **lambda** █ = manejo de **casos borde** █ = solución en **una sola pasada (O(n))**

I CONDICIONALES · PREGUNTAS 1 – 5

- 1 `clasificar_balance(precision, recall)` → "Crítico" / "Desbalanceado" / "Óptimo" / "Aceptable". Valide rango [0,1].
- 2 Con `resultados` (prob, etiqueta) y umbral 0.5, imprima TP/TN/FP/FN por caso con un `for`.
- 3 `detectar_tendencia(valores)` → "ascendente"/"descendente"/"mixta"/"sin datos". Una sola pasada.
- 4 De un dict anidado de modelos, imprima el **activo** con mayor `f1` o "Sin modelos activos".
- 5 `evaluar_umbral(valor, umbral, tolerancia=0.01)` → "sobre"/"bajo"/"en rango". Maneje `None`.

II CICLOS FOR · PREGUNTAS 6 – 10

- 6 Con `precios`, calcule media, desviación estándar poblacional y outliers ($> 1\sigma$) en una pasada.
- 7 Agrupe montos por moneda en un dict a partir de una lista de transacciones.
- 8 Transponga `[[1,2,3],[4,5,6]]` y multiplique por 2 → `[[2,8],[4,10],[6,12]]`.
- 9 Sobre `serie`, en una pasada: máx, mín, índices y cantidad de valores repetidos.
- 10 Cuente frecuencia de palabras en un texto y muestre las 3 más frecuentes.

III CICLOS WHILE · PREGUNTAS 11 – 15

- 11 Encuentre el primer primo > 1000 con `while`. Cuente iteraciones.
- 12 Invierta los dígitos de `123456789` con `while` (sin `str`).
- 13 Simule convergencia tipo K-Means hasta `|nuevo - centroide| ≤ 0.001`.
- 14 Calcule la mediana sin `sorted` ni `statistics` usando extracción repetida del mínimo.
- 15 Monte Carlo: 10000 puntos, estime π con `4·dentro/N`. Reporte error absoluto.

λ 16 `varianza(lista)` con `def` (una pasada) y `varianza_1 = lambda ...`. `None` si `len < 2`.

λ 17 Ordene tuplas `(nombre, precision, recall)` por F1-score descendente con `lambda`. Maneje división por cero.

λ 18 Con una sola comprensión + `lambda`, filtre positivos pares y elévelos al cuadrado.

λ 19 Con `lambda`, `map` y `reduce`, calcule el producto punto de dos vectores.

λ 20 `crear_evaluador(umbrales)` que retorne una `lambda` clasificando en "Q1..Q4".

Rúbrica y Soluciones — v4

Documento para el docente · uso interno

 Confidencial

✓ RÚBRICA DE EVALUACIÓN

CRITERIO	PUNTOS
Sintaxis correcta y uso adecuado de estructuras	0.2
Lógica correcta del algoritmo	0.2
Manejo de casos borde (listas vacías, división por cero, NaN)	0.1
Total por pregunta	0.5

Nota sobre ⚡ : las preguntas marcadas exigen una solución en **una sola pasada**; las que usen dos o más pasadas reciben máximo 0.3 pts.

💡 SOLUCIONES BREVES (REFERENCIA)

```
# 1
def clasificar_balance(p, r):
    if not (0<=p<=1 and 0<=r<=1):
        return "entrada inválida"
    if p < 0.5 or r < 0.5: return "Crítico"
    if abs(p - r) > 0.2: return "Desbalanceado"
    if p >= 0.8 and r >= 0.8: return "Óptimo"
    return "Aceptable"

# 3 — una pasada
def detectar_tendencia(v):
    if len(v) < 2: return "sin datos"
    asc = desc = True
    for i in range(len(v)-1):
        if v[i] >= v[i+1]: asc = False
        if v[i] <= v[i+1]: desc = False
    return "ascendente" if asc else "descendente" if desc else "mixto"

# 5
def evaluar_umbral(v, u, t=0.01):
    if v is None or u is None: return "entrada inválida"
    if v > u + t: return "sobre"
    if v < u - t: return "bajo"
    return "en rango"

# 6 — una pasada (WelFord)
precios = [100,120,80,200,150,90,300]
n = 0; media = 0.0; M2 = 0.0
for x in precios:
    n += 1
    delta = x - media
    media += delta / n
    M2 += delta * (x - media)
var = M2 / n
sd = var ** 0.5
out = sum(1 for x in precios if abs(x - media) > sd)
print(media, sd, out)

# 7
tx = [
    {"id":1,"monto":250,"moneda":"USD"},
    {"id":2,"monto":180,"moneda":"EUR"},
    {"id":3,"monto":420,"moneda":"USD"},
    {"id":4,"monto":90,"moneda":"COP"},
    {"id":5,"monto":310,"moneda":"USD"},
```

```
]
tot = {}
for t in tx:
    tot[t["moneda"]] = tot.get(t["moneda"], 0) + t["monto"]

# 9 — una pasada
s = [3,1,4,1,5,9,2,6,5,3,5]
mx = mn = s[0]; ix = inx = 0; fr = {}
for i, v in enumerate(s):
    if v > mx: mx, ix = v, i
    if v < mn: mn, inx = v, i
    fr[v] = fr.get(v, 0) + 1
rep = sum(1 for c in fr.values() if c >= 2)

# 10
texto = "machine learning es una rama de la inteligencia artificial"
fr = {}
for p in texto.split():
    fr[p] = fr.get(p, 0) + 1
top3 = sorted(fr.items(), key=lambda x: -x[1])[0:3]

# 11
def es_primo(n):
    if n < 2: return False
    i = 2
    while i*i <= n:
        if n % i == 0: return False
        i += 1
    return True

c, it = 1001, 0
while True:
    it += 1
    if es_primo(c): break
    c += 1

# 12
n, inv = 123456789, 0
while n > 0:
    inv = inv * 10 + n % 10
    n //= 10

# 13
c, nv, it = 0.0, 10.0, 0
while abs(nv - c) > 0.001:
    c, nv = nv, (nv + 5) / 2
```

```

it += 1

# 14 - mediana sin sorted
datos = [12,7,3,9,15,2,8,4]
copia = datos[:]
def saca_min(l):
    k = 0
    for i in range(1, len(l)):
        if l[i] < l[k]: k = i
    return l.pop(k)
m = len(copia) // 2
for _ in range(m): saca_min(copia)
mediana = saca_min(copia) if len(datos) % 2 \
    else (saca_min(copia) + saca_min(copia)) / 2

# 15
import random
N, dentro = 10000, 0
for _ in range(N):
    x, y = random.random(), random.random()
    if x*x + y*y <= 1: dentro += 1
pi = 4 * dentro / N

# 16
def varianza(l):
    if len(l) < 2: return None
    n = len(l); m = 0.0; M2 = 0.0
    for i, x in enumerate(l, 1):
        d = x - m
        m += d / i
        M2 += d * (x - m)

varianza_1 = lambda l: None if len(l) < 2 else \
    sum((x - sum(l)/len(l))**2 for x in l) / len(l)

# 17
modelos = [("A",0.8,0.9),("B",0.6,0.6),("C",0.9,0.7)]
orden = sorted(modelos,
    key=lambda t: 0 if (t[1]+t[2])==0
        else 2*t[1]*t[2]/(t[1]+t[2]),
    reverse=True)

# 18
nums = [-2,3,4,-5,6,7,8]
res = [(lambda x: x*x)(x) for x in nums
    if x > 0 and x % 2 == 0]

# 19
from functools import reduce
v1 = [1,2,3,4]; v2 = [5,6,7,8]
prod = reduce(lambda a, b: a + b,
    map(lambda p: p[0]*p[1], zip(v1, v2)))

# 20
def crear_evaluador(umbrales):
    return lambda v: next(
        (f"Q{i+1}" for i, u in enumerate(umbrales)
            if v < u),
        f"Q{len(umbrales)+1}")

ev = crear_evaluador([0.25, 0.5, 0.75])
print(ev(0.1), ev(0.6), ev(0.9))

```